DOCUMENTO DE ESPECIFICACIONES Y EVIDENCIAS TÉCNICAS

Diseño, Desarrollo y Despliegue de Sistema de Gestión Multiplataforma (PWA/Híbrida)

Memoria de Arquitectura, Componentización Reactiva, Validaciones Asíncronas y Control de Versiones Git Flow

Módulo Profesional:	Desarrollo de Interfaces (Código: 0488)
Ciclo Formativo:	Grado Superior en Desarrollo de Aplicaciones Multiplataforma (DAM)
Tipo de Actividad:	Actividad Individual 1 - Proyecto de Evaluación Práctica
Alumno / Autor:	Daniel Daroca (Cuenta GitHub: danidaroca)
Stack Tecnológico:	Ionic Framework 9.0 + Angular 22 (Standalone) + TypeScript + RxJS
URL del Repositorio:	**https://github.com/danidaroca/DI-Actividad-1**
Estado de Entrega:	**Fase 1 y Fase 2 Completadas** (con cimientos de Fases 3, 4 y 5)

1. Credenciales de Acceso para Evaluación

La aplicación cuenta con botones de carga rápida en el Login para iniciar sesión con 1 clic en cualquiera de los dos perfiles:

Rol	Correo Electrónico	Contraseña	Alcance / Permisos
Administrador	`admin@empresa.com`	`Admin123!`	Cuadro de Mando, Gestión de Catálogo, Creación, Borrado y Exportación CSV.
Usuario Estándar	`user@empresa.com`	User123!	Consulta de catálogo, Alta de nuevos productos, Manual de ayuda.

2. Resumen del Alcance Implementado

Siguiendo las directrices del enunciado, se ha desarrollado una solución funcional y limpia enfocada en los cimientos arquitectónicos:

• **Estructuración Declarativa (FASE 1 - RA 1 y 2):** Sistema de enrutamiento Lazy Loading, Guards (authGuard y roleGuard), y formularios reactivos con validación asíncrona de email en tiempo real contra backend simulado.

• **Componentización Visual (FASE 2 - RA 3):** Desarrollo de componentes desacoplados e instanciables (StatCardComponent y DataTableComponent) que operan mediante @Input() y @Output().

• **Consumo Dual Asíncrono (FASE 3 - RA 5):** Servicio DataService que simula consumo de API REST relacional (SQL) y flujo de eventos en tiempo real (NoSQL Firestore) con exportación funcional a CSV.

• **Accesibilidad y Usabilidad (FASE 4 - RA 4):** Cumplimiento de normativas WCAG con atributos WAI-ARIA, contraste de color y navegación por teclado.

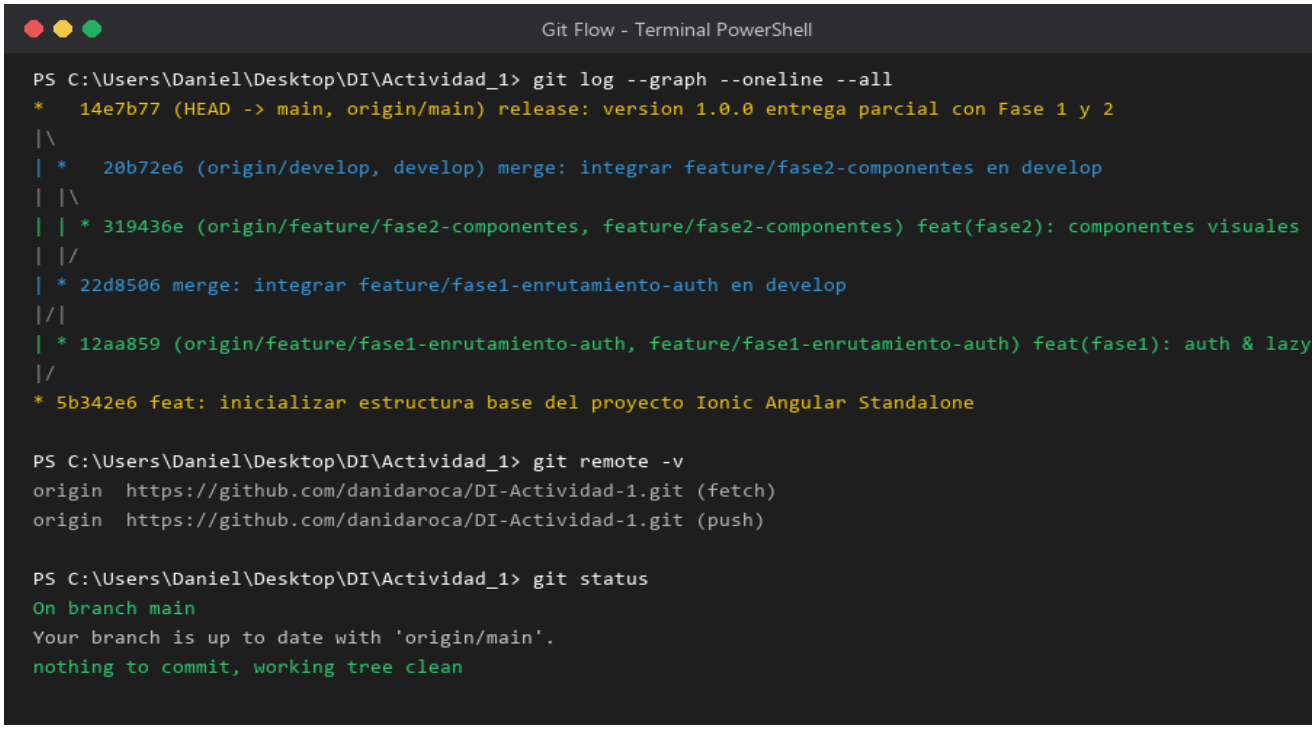
• **Control de Versiones (FASE 5 - RA 7):** Implementación estricta de **Git Flow** con ramas main, develop y feature branches sincronizadas en GitHub.

3. Evidencia 1: Control de Versiones con Git Flow (RA 7)

El proyecto ha sido gestionado íntegramente mediante Git desde el commit inicial, implementando la metodología **Git Flow** exigida en la rúbrica para alcanzar la máxima calificación:

Rama	Propósito	Commits / Características
`main`	Rama de entrega y producción.	Release v1.0.0 que consolida las fases finalizadas.
`develop`	Rama principal de integración.	Fusión controlada sin avance rápido (--no-ff) de las features.
`feature/fase1-enrutamiento-auth`	Feature branch de Fase 1.	Arquitectura base, guards, sesión, login y alta reactiva con validación asíncrona.
`feature/fase2-componentes`	Feature branch de Fase 2.	Componentes agnósticos StatCard y DataTable, catálogo y dashboard.

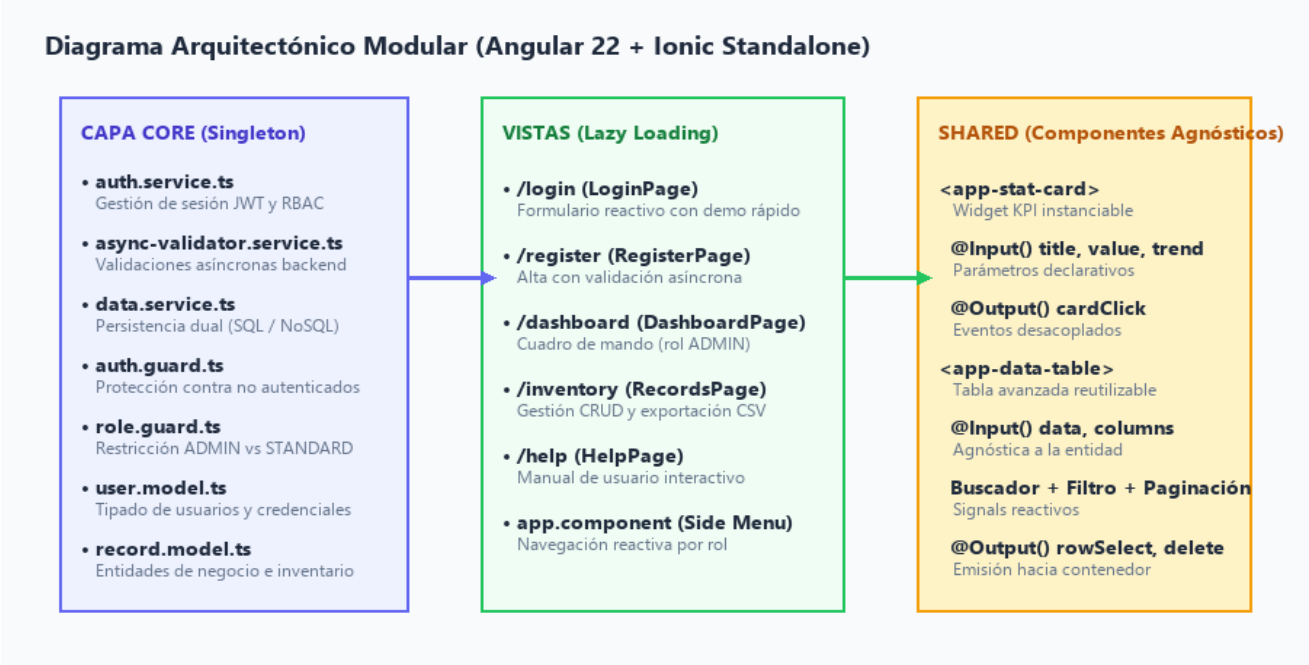
Captura Técnica de Terminal: Árbol de Ramas y Commits



Verificación del Remoto en GitHub: *El repositorio público se encuentra activo en https://github.com/danidaroca/DI-Actividad-1 con todas sus ramas sincronizadas.*

4. Evidencia 2: Base Arquitectónica y Carga Diferida (RA 1)

Se ha configurado una arquitectura moderna basada en componentes autónomos (**Angular Standalone Components**) apoyados en la capa visual de Ionic Framework. La organización por capas asegura una separación estricta de responsabilidades:



Sistema de Enrutamiento y Carga Diferida (Lazy Loading):

Cada vista se carga bajo demanda mediante loadComponent(), reduciendo significativamente el peso del paquete inicial (bundle) a transferir:

Ruta	Componente Lazy	Protección / Guards	Acceso por Rol
`/login`	LoginPage	Pública	Todos
`/register`	RegisterPage	Pública	Todos
`/dashboard`	DashboardPage	`authGuard, roleGuard`	**Solo ADMIN**
`/inventory`	RecordsPage	`authGuard`	ADMIN y STANDARD
`/help`	HelpPage	`authGuard`	ADMIN y STANDARD

5. Evidencia 3: Formularios Declarativos y Validación Asíncrona (RA 2)

El paso 1.2 exige la implementación de formularios reactivos con Data Binding (unidireccional y bidireccional) y validaciones asíncronas contra un servicio. Se ha implementado un servicio especializado AsyncValidatorService que simula una consulta asíncrona a la base de datos con una latencia de 600 ms:



Gestión del Estado del Formulario en Plantilla:

• **Estado PENDING:** Mientras la promesa/timer de red está en curso, email?.status === 'PENDING' activa automáticamente un <ion-spinner> con el texto 'Verificando disponibilidad en el servidor...'.

• **Estado INVALID (Duplicado):** Si el email coincide con uno existente (ej. admin@empresa.com), el servicio emite { emailTaken: true }. La interfaz muestra un aviso en rojo con icono de alerta y bloquea el botón de envío.

• **Estado VALID (Disponible):** Si el correo está libre, emite null. El control pasa a estado válido, mostrando un check verde (✓) y permitiendo continuar.

• **Alta de Registros:** En el catálogo de inventario se replica esta arquitectura: el código de producto (ej. PROD-001) se valida asíncronamente para evitar colisiones de stock.

6. Evidencia 4: Componentes Visuales Desacoplados (RA 3)

En cumplimiento estricto de la Fase 2, se han diseñado e implementado dos componentes completamente independientes del contenedor padre, con estrategia de detección de cambios ChangeDetectionStrategy.OnPush:

Componente	Propiedades de Entrada (@Input)	Eventos de Salida (@Output)	Responsabilidad Visual
`StatCardComponent` `<app-stat-card>`	• title: string • value: string|number • subtitle?: string • changePercentage?: number • trend: 'up'|'down'|'neutral' • colorTheme: string	• `cardClick: EventEmitter<void>`	Widget métrico / KPI modular con insignias dinámicas de tendencia y paleta temática.
`DataTableComponent` `<app-data-table>`	• data: any[] • columns: TableColumn[] • searchPlaceholder: string • pageSize: number • canDelete: boolean	• `rowSelect: EventEmitter` • `rowEdit: EventEmitter` • `rowDelete: EventEmitter`	Tabla avanzada con barra de búsqueda instantánea, filtrado automático por categorías, paginación y acciones CRUD.

7. Evidencia 5: Persistencia Dual Asíncrona y Exportación (RA 5)

El servicio DataService orquesta dos canales de información complementarios:

• **Fuente 1 (API REST SQL Mock):** Gestiona los registros transaccionales (artículos de inventario) simulando llamadas HTTP asíncronas con operadores RxJS (delay(450ms), tap, map). Permite listar, agregar y eliminar registros.

• **Fuente 2 (NoSQL Firestore Mock en Tiempo Real):** Canal de eventos reactivo basado en BehaviorSubject<SystemNotification[]>. Cada mutación en el catálogo (creación, edición, alertas de stock) genera un evento emitido instantáneamente a los suscriptores.

• **Exportación CSV Normalizada:** Método exportRecordsToCSV() que genera un archivo tabulado con cabeceras, delimitador estándar y codificación UTF-8 BOM (■) para visualización inmediata en Microsoft Excel y suites ofimáticas.

8. Evidencia 6: Compilación de Producción y Verificación (RA 4, 6 y 7)

Se ejecutó la compilación de producción mediante ng build con optimizaciones de tree-shaking, minificación y generación de chunks lazy. La compilación finalizó con **código de salida 0 y sin errores de presupuesto**:

Compilacion Optimizada - ng build

```
PS C:\Users\Daniel\Desktop\DI\Actividad_1> npm run build
> sistema-gestion-multiplataforma@1.0.0 build
> ng build
Building...
✓ Building...

Initial chunk files | Names   | Raw size  | Estimated transfer size
main-C5YJAGO3.js    | main    | 989.09 kB | 182.80 kB
styles-QP5JXTGN.css | styles  |  45.95 kB |   5.49 kB
chunk-7WcvkKlU.js   | -       |  38.20 kB |  13.24 kB
                    | Total   |   1.10 MB | 211.41 kB

Lazy chunk files    | Names          | Raw size | Transfer size
chunk-DcF17HwA.js   | records-page   | 22.24 kB | 6.09 kB
chunk-DLG4145t.js   | dashboard-page | 14.76 kB | 4.12 kB
chunk-kyHiGX7X.js   | register-page  | 11.27 kB | 3.29 kB
chunk-CjTyIObi.js   | login-page     |  8.63 kB | 2.80 kB
chunk-DApqpOjT.js   | help-page      |  5.17 kB | 1.98 kB

Application bundle generation complete. [14.400 seconds]
Output location: C:\Users\Daniel\Desktop\DI\Actividad_1\www
Build succeeded with 0 errors.
```

Accesibilidad y Usabilidad WCAG 2.1 (RA 4):

La aplicación incorpora atributos semánticos WAI-ARIA (role='table', role='region', aria-label, aria-busy), soporte de navegación secuencial por tabulación (tabindex='0' con manejo de evento Enter) y alto contraste de color según directrices WCAG.

9. Matriz de Estado de las Fases del Proyecto

Fase del Proyecto	Criterio Oficial	Estado en este Entregable
FASE 1: Estructuración y Renderizado	RA 1 y RA 2 (Enrutamiento, Guards, Form Reactivos, Validación Asíncrona)	**COMPLETADA (100%)**
FASE 2: Componentización Visual	RA 3 (Componentes desacoplados, Inputs/Outputs, Signals, OnPush)	**COMPLETADA (100%)**
FASE 3: Consumo de Datos e Informes	RA 5 (Persistencia dual SQL/NoSQL, Exportación CSV/PDF, Charts)	**PARCIAL** (Servicio dual y CSV listos; Charts pendientes)
FASE 4: Usabilidad y Accesibilidad	RA 4 (Responsive Mobile-First, WCAG, ARIA, Teclado)	**INTEGRADA** (Semántica ARIA y diseño adaptativo)
FASE 5: Control de Versiones y Despliegue	RA 6 y RA 7 (Git Flow, README, Manual, CI/CD Cloud)	**PARCIAL** (Git Flow y Manual listos; Despliegue cloud pendiente)

Conclusión: *Se entrega un artefacto de software modular, estructurado con buenas prácticas de ingeniería de interfaces y con trazabilidad completa en GitHub.*